UNICURITIBA
CURSO: [PREENCHER O NOME DO CURSO]
COMPUTAÇÃO GRÁFICA E REALIDADE VIRTUAL

Gabriel Thiago Casini Keppen (RA 172422353)
Felipe Silva Muchinski (RA 172422440)

SANDBOX GRÁFICO INTERATIVO PARA VISUALIZAÇÃO DE ILUMINAÇÃO E CONCEITOS MATEMÁTICOS EM COMPUTAÇÃO GRÁFICA UTILIZANDO UNREAL ENGINE

Entrega parcial do Projeto A3

Curitiba
2026

Gabriel Thiago Casini Keppen
Felipe Silva Muchinski

SANDBOX GRÁFICO INTERATIVO PARA VISUALIZAÇÃO DE ILUMINAÇÃO E CONCEITOS MATEMÁTICOS EM COMPUTAÇÃO GRÁFICA UTILIZANDO UNREAL ENGINE

Documento referente à entrega parcial do Projeto A3 da unidade curricular Computação Gráfica e Realidade Virtual, apresentado como requisito de avaliação.

Professor: Rubem Matimoto Koide

Curitiba
2026

SUMÁRIO

1 Introdução 4
1.1 Contextualização 4
1.2 Problema e proposta 4
1.3 Objetivos 4
1.4 Justificativa 5
1.5 Metodologia 5
2 Revisão bibliográfica 5
2.1 Iluminação difusa e o modelo de Lambert 5
2.2 Vetores, normal e produto escalar 6
2.3 Unreal Engine e Blueprints 6
2.4 Ferramentas de apoio ao desenvolvimento 6
3 Desenvolvimento parcial 7
3.1 Ambiente de desenvolvimento e organização do projeto 7
3.2 Cena de testes 7
3.3 Blueprint BP_LightAnalyzer 7
3.4 Resultados parciais e validação 9
3.5 Limitações e próximas etapas 10
Referências 10

1 INTRODUÇÃO

1.1 Contextualização

A computação gráfica utiliza, de forma contínua, conceitos matemáticos para representar, transformar e iluminar objetos em ambientes tridimensionais. Operações com vetores, matrizes, transformações geométricas e produtos escalares estão presentes em cada quadro renderizado (HUGHES et al., 2013; AKENINE-MÖLLER; HAINES; HOFFMAN, 2018). Nos motores gráficos atuais, no entanto, boa parte desse cálculo ocorre de maneira automática e fica oculta de quem desenvolve ou estuda a área, o que dificulta relacionar a teoria vista em sala com o resultado visual obtido.

1.2 Problema e proposta

Como tornar perceptível, em tempo real, a relação entre a matemática da iluminação e a aparência final de uma superfície em um motor gráfico? Para responder a essa pergunta, o projeto propõe um **sandbox gráfico interativo** desenvolvido na Unreal Engine, no qual o usuário manipula uma fonte de luz e observa, sobre a própria cena, o vetor normal da superfície, a direção da luz, o produto escalar entre eles, o ângulo de incidência e a intensidade difusa resultante.

1.3 Objetivos

Objetivo geral: desenvolver um ambiente 3D interativo que permita visualizar, em tempo real, os cálculos vetoriais envolvidos na iluminação difusa de uma superfície.

Objetivos específicos:

- construir uma cena 3D na Unreal Engine com superfície, piso e fonte de luz manipulável;
- implementar em Blueprint o cálculo do produto escalar $N \cdot L$, do ângulo de incidência e da intensidade difusa;
- representar graficamente os vetores normal e de direção da luz diretamente na cena;
- validar os valores calculados pelo Blueprint contra o cálculo teórico;

• projetar a estrutura de modo modular, permitindo incluir futuramente reflexão especular, transformações, matrizes, materiais e sombras.

1.4 Justificativa

O projeto tem valor didático: ao expor valores normalmente invisíveis, aproxima o formalismo matemático da experiência visual e funciona como ferramenta de apoio ao estudo da disciplina. Também se alinha às aplicações de computação gráfica e realidade virtual discutidas em aula, como visualização técnica, simulação e treinamento (KOIDE, 2026).

1.5 Metodologia

O desenvolvimento segue abordagem incremental, partindo de um produto mínimo viável (cena, luz manipulável, visualização dos vetores e cálculo da iluminação difusa) e incorporando módulos conforme o tempo permite. O trabalho é versionado em repositório Git hospedado no GitHub. Na criação do Blueprint e da cena foi utilizado um assistente de inteligência artificial (Claude Code), conectado ao editor por meio do Model Context Protocol, como apoio à automação de tarefas do editor; o projeto, as decisões de modelagem e a validação dos resultados foram conduzidos pela equipe.

2 REVISÃO BIBLIOGRÁFICA

2.1 Iluminação difusa e o modelo de Lambert

Superfícies foscas espalham a luz recebida de maneira aproximadamente igual em todas as direções. Essa propriedade é descrita pela lei do cosseno de Lambert, segundo a qual a luz refletida por um ponto da superfície é proporcional ao cosseno do ângulo entre a normal da superfície e a direção da luz incidente (LAMBERT, 1760). Na computação gráfica, esse é o modelo de reflexão difusa mais elementar e constitui o termo difuso de modelos de iluminação local mais completos (HUGHES et al., 2013). Em sua forma usual:

$$I_d = I_L \cdot k_d \cdot \max(0, N \cdot L) \qquad (1)$$

em que I_d é a intensidade difusa resultante, I_L a intensidade da luz, k_d o coeficiente de reflexão difusa do material, N o vetor normal unitário da superfície e L o vetor

unitário que aponta do ponto da superfície para a fonte de luz. A função máx impede contribuição negativa quando a luz incide pela face oposta da superfície.

2.2 Vetores, normal e produto escalar

O produto escalar entre dois vetores a e b é definido como $a \cdot b = |a|\,|b| \cos \theta$, em que θ é o ângulo entre eles. Para vetores unitários, o módulo é igual a 1 e o produto escalar coincide com o cosseno do ângulo (AKENINE-MÖLLER; HAINES; HOFFMAN, 2018). Assim, o ângulo de incidência da luz pode ser recuperado por:

$$\theta = \arccos(N \cdot L) \qquad (2)$$

Para uma luz pontual na posição P_L e um ponto da superfície na posição P, o vetor de direção da luz é obtido pela normalização da diferença entre as posições:

$$L = (P_L - P) / |P_L - P| \qquad (3)$$

2.3 Unreal Engine e Blueprints

A Unreal Engine, desenvolvida pela Epic Games, é um motor gráfico de tempo real amplamente utilizado em jogos, visualização arquitetônica, simulação e produção virtual. Seu sistema de programação visual, o Blueprint, permite construir lógica de interação por meio de nós conectados, sem escrever código tradicional, com suporte a C++ quando necessário (EPIC GAMES, 2026). Esse recurso foi escolhido por permitir iterar rapidamente sobre o cálculo e por tornar a lógica legível para fins didáticos, conforme as ferramentas apresentadas em aula (KOIDE, 2026).

2.4 Ferramentas de apoio ao desenvolvimento

Git, GitHub e Git LFS. O controle de versão é feito com Git (CHACON; STRAUB, 2014), com repositório remoto no GitHub. Como projetos Unreal possuem arquivos binários grandes (.uasset e .umap), é utilizado o Git LFS, que armazena esses arquivos fora do histórico regular do repositório.

Model Context Protocol (MCP). O MCP é um protocolo aberto que padroniza a conexão entre assistentes de inteligência artificial e ferramentas externas (MODEL CONTEXT PROTOCOL, 2026). A Unreal Engine 5.8 inclui, como recurso experimental, o plugin ModelContextProtocol, que faz o editor atuar como servidor MCP e expõe ferramentas para manipular atores, níveis, assets e grafos de

Blueprint. No projeto, esse plugin foi ativado (juntamente com os toolsets EditorToolset e UMGToolSet) para automatizar a criação da cena e do Blueprint.

3 DESENVOLVIMENTO PARCIAL

3.1 Ambiente de desenvolvimento e organização do projeto

O projeto foi criado na Unreal Engine 5.8 com o nome LightSandbox, a partir de um modelo em branco baseado em Blueprint. O código-fonte e os assets são versionados no repositório privado da equipe no GitHub (github.com/Gabrielkeppen/Light_SandBox), com Git LFS habilitado para os assets binários. O Quadro 1 resume o ambiente.

Quadro 1 – Ambiente de desenvolvimento

Item	Descrição
Motor gráfico	Unreal Engine 5.8
Linguagem / lógica	Blueprint (programação visual)
Plugins ativados	ModelContextProtocol, EditorToolset, UMGToolSet, MCPClientToolset
Controle de versão	Git, GitHub (repositório privado) e Git LFS
Sistema operacional	Windows 10

Fonte: Elaborado pelos autores (2026).

3.2 Cena de testes

A cena de testes (nível SandBoxMap) é composta por três elementos, descritos no Quadro 2: um piso, uma luz pontual e o ator analisador, que representa a superfície cujo vetor normal e iluminação são estudados.

Quadro 2 – Elementos da cena

Elemento	Tipo	Posição (cm)	Observações
Chao	Static Mesh (plano)	(0, 0, 0)	Escala 20 × 20; referência visual
LuzPontual	Point Light	(350, −250, 400)	Fonte de luz manipulável
Analisador	BP_LightAnalyzer	(0, 0, 150)	Inclinado 20° (pitch); referencia a LuzPontual

Fonte: Elaborado pelos autores (2026).

3.3 Blueprint BP_LightAnalyzer

O Blueprint BP_LightAnalyzer é um ator que contém uma malha em forma de cubo achatado (4 × 4 × 0,1 unidades de escala) usada como superfície. As variáveis do Blueprint estão no Quadro 3.

Quadro 3 – Variáveis do Blueprint

Variável	Tipo	Função
LightActor	Referência a ator	Luz analisada; editável por instância na cena
NdotL	Ponto flutuante	Produto escalar N · L
AngleDeg	Ponto flutuante	Ângulo de incidência θ, em graus
DiffusePct	Ponto flutuante	Intensidade difusa, em porcentagem

Fonte: Elaborado pelos autores (2026).

A cada quadro (evento Tick), e desde que a referência à luz seja válida, o Blueprint executa os passos abaixo, que implementam diretamente as equações 1 a 3:

1) obtém a posição do ator (P) e a posição da luz (P_L);

2) calcula L = normalizar(P_L − P);

3) obtém a normal N como o vetor “para cima” (Up Vector) do ator;

4) calcula N · L com o nó Dot Product;

5) calcula θ = arccos(N · L), em graus, com o nó Acos (Degrees);

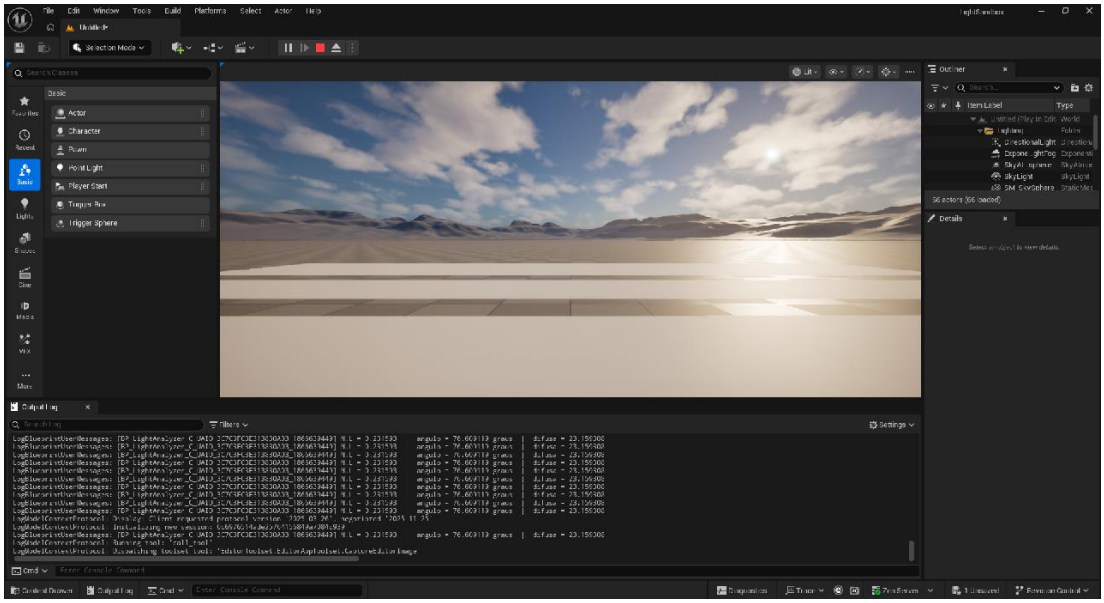
6) calcula a intensidade difusa como máx(0, N · L) × 100;

7) desenha, com Draw Debug Arrow, a seta verde da normal e a seta amarela da direção da luz;

8) exibe os valores na tela e no log (Print String).

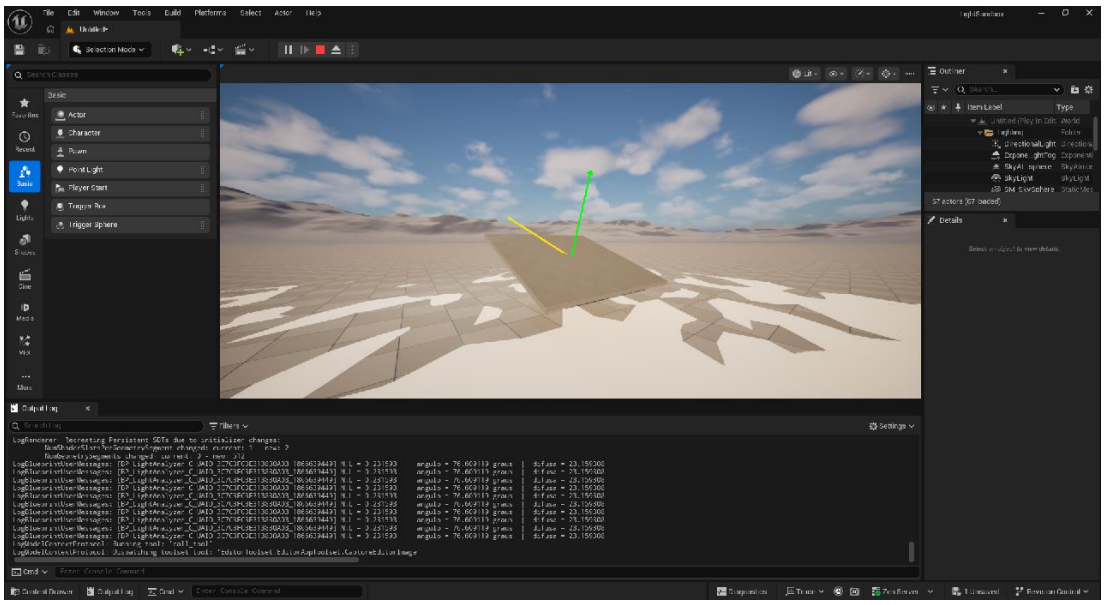
A Figura 1 mostra o ambiente com a simulação em execução e o log com os valores calculados. A Figura 2 apresenta a superfície inclinada com as setas desenhadas: em verde a normal N e, em amarelo, a direção da luz L.

Figura 1 – Editor da Unreal Engine com simulação em execução e log do cálculo



Fonte: Elaborado pelos autores (2026).

Figura 2 – Superfície analisada com a normal (verde) e a direção da luz (amarelo)



Fonte: Elaborado pelos autores (2026).

3.4 Resultados parciais e validação

Para validar a implementação, a luz foi posicionada em cinco pontos distintos e os valores exibidos pelo Blueprint foram comparados com o cálculo teórico das equações 1 a 3, feito de forma independente. Com o analisador em (0, 0, 150) e inclinado 20°, a normal vale N = (−0,342; 0; 0,940). O Quadro 4 apresenta os resultados.

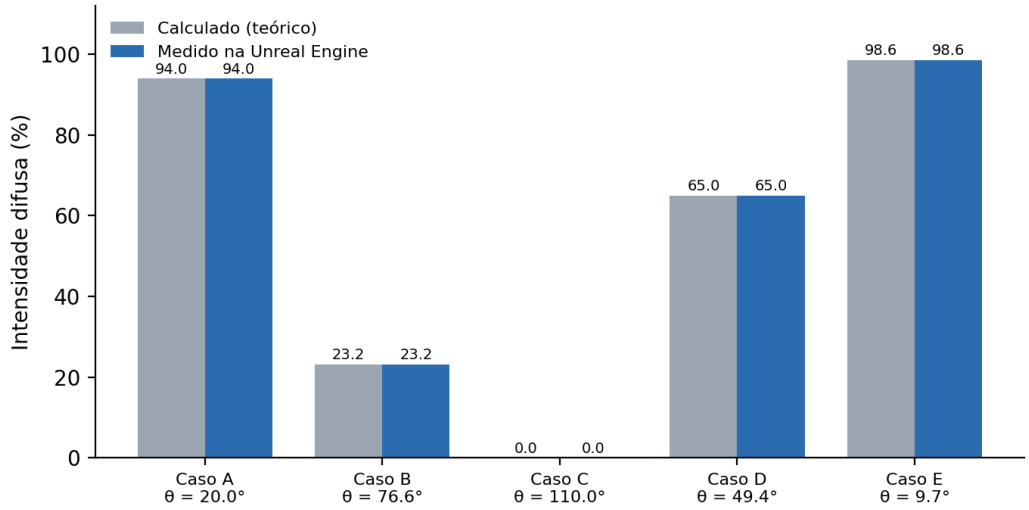
Quadro 4 – Comparação entre valores teóricos e medidos

Caso	Posição da luz (cm)	N·L teórico	N·L medido	θ teórico	θ medido	Difusa medida
A	(0, 0, 550)	0,9397	0,9397	20,0°	20,0°	94,0%
B	(350, -250, 400)	0,2316	0,2316	76,6°	76,6°	23,2%
C	(400, 0, 150)	-0,3420	-0,3420	110,0°	110,0°	0,0%
D	(-400, 0, 300)	0,6502	0,6502	49,4°	49,4°	65,0%
E	(-200, 0, 500)	0,9856	0,9856	9,7°	9,7°	98,6%

Fonte: Elaborado pelos autores (2026).

Em todos os casos os valores medidos coincidiram com os teóricos até a sexta casa decimal, o que valida a implementação do cálculo. O caso C ilustra o comportamento do termo máx(0, N · L): a luz incide pela face oposta (N · L = −0,342; θ = 110°) e, por isso, a intensidade difusa é 0%. A Figura 3 resume a comparação para a intensidade difusa.

Figura 3 – Intensidade difusa calculada e medida em cada caso



Fonte: Elaborado pelos autores (2026).

3.5 Limitações e próximas etapas

O que está implementado cobre o produto mínimo viável do módulo de iluminação difusa. Permanecem como próximas etapas: (a) interface gráfica (UMG) com os valores em texto sobre a tela, em substituição ao texto de depuração; (b) controles para o usuário mover a luz durante a execução; (c) inclusão do coeficiente do material (k_d) e da intensidade da luz (I_L) no cálculo da equação 1; (d) suporte a luz direcional, além da pontual; e (e) novos módulos, como reflexão especular, transformações e matrizes, sombras e propriedades de materiais. Como limitação atual, a normal é obtida do vetor “para cima” do ator, o que é adequado a superfícies planas, mas não considera malhas com normais variáveis por vértice.

REFERÊNCIAS

AKENINE-MÖLLER, Tomas; HAINES, Eric; HOFFMAN, Naty. **Real-time rendering**. 4. ed. Boca Raton: CRC Press, 2018.

CHACON, Scott; STRAUB, Ben. **Pro Git**. 2. ed. Berkeley: Apress, 2014. Disponível em: https://git-scm.com/book. Acesso em: 12 out. 2026.

EPIC GAMES. **Unreal Engine documentation**. 2026. Disponível em: https://dev.epicgames.com/documentation/unreal-engine. Acesso em: 12 out. 2026.

HUGHES, John F. et al. **Computer graphics**: principles and practice. 3. ed. Upper Saddle River: Addison-Wesley, 2013.

KOIDE, Rubem Matimoto. **Computação gráfica e realidade virtual**: aula 1. Curitiba: UniCuritiba, 2026. Material de aula.

LAMBERT, Johann Heinrich. **Photometria sive de mensura et gradibus luminis, colorum et umbrae**. Augsburg: Eberhard Klett, 1760.

MODEL CONTEXT PROTOCOL. **Specification**. 2026. Disponível em: https://modelcontextprotocol.io. Acesso em: 12 out. 2026.